

Segurança em Aplicações Web: Vulnerabilidades Comuns e Defesas

Ana da Silva, Fabricio Fernandes, Matheus Gussiardi, Matheus Izidio, Ubiratã Costa

¹ Centro de Matemática, Computação e Cognição (CMCC)
Universidade Federal do ABC (UFABC)
Av. dos Estados, 5001, Bangu, Santo André/SP, Brazil

Resumo. *Este trabalho analisa vulnerabilidades críticas em sistemas web baseadas no OWASP Top 10 [None 2008], utilizando a defesa em profundidade como princípio unificador para integrar ferramentas de análise (SAST, DAST, IAST) e proteção em runtime (RASP) [Shahriar and Zulkernine 2011, De Jesús Domínguez-García et al. 2023, Barth et al. 2022]. A metodologia consistiu em levantamento bibliográfico via Google Scholar com suporte de IA generativa para síntese conceitual [De Jesús Domínguez-García et al. 2023]. Identificaram-se lacunas na literatura sobre a segurança de shadow APIs [Escape 2023, Tanveer 2025] e a interpretabilidade de IA na detecção de ameaças [Yadav 2025a, Alsaedi 2023]. Conclui-se que o futuro da área reside na automação via DevSecOps [Edmundson and Hartman 2022] e na consolidação de arquiteturas Zero Trust [of Standards and (NIST) 2020, Yadav 2025b].*

Palavras-chave: *Segurança Web, Defesa em Profundidade, DevSecOps, Zero Trust.*

Abstract. *This study analyzes critical web vulnerabilities based on the OWASP Top 10 [None 2008], using defense-in-depth as a unifying principle to integrate analysis tools (SAST, DAST, IAST) and runtime protection (RASP) [Shahriar and Zulkernine 2011, De Jesús Domínguez-García et al. 2023, Barth et al. 2022]. The methodology involved a literature review via Google Scholar supported by generative AI for conceptual synthesis [De Jesús Domínguez-García et al. 2023]. Gaps were identified regarding shadow API security [Escape 2023, Tanveer 2025] and the interpretability of AI in threat detection [Yadav 2025a, Alsaedi 2023]. The study concludes that the field's future lies in DevSecOps automation [Edmundson and Hartman 2022] and the consolidation of Zero Trust architectures [of Standards and (NIST) 2020, Yadav 2025b].*

Keywords: *Web Security, Defense-in-Depth, DevSecOps, Zero Trust.*

1. Introdução

A transformação digital impulsionou drasticamente a demanda por aplicativos web e mobile amigáveis e interativos, aumentando significativamente a base de usuários desses serviços.

Essa digitalização massiva expandiu enormemente a superfície de ataque: uma vez que as aplicações web armazenam e trafegam grande quantidade de dados sensíveis

e lógica de negócios, explorar suas vulnerabilidades passa a ser atrativo para invasores. Logo, a segurança destas aplicações tornou-se um pilar estratégico para as organizações.

Devido à natureza dinâmica da web, as aplicações dependem constantemente de entradas de dados fornecidas pelos usuários. A falha na validação ou sanitização adequada desses dados expõe o sistema a vulnerabilidades graves. Historicamente, as estratégias de defesa focavam na camada de rede; entretanto, atacantes têm priorizado a camada de aplicação, para contornar perímetros de segurança e acessar dados valiosos diretamente na fonte.

Neste contexto, o presente trabalho explora os conceitos fundamentais de segurança em aplicações web, abordando as vulnerabilidades mais críticas — como injeção de código, XSS e falhas de controle de acesso —, apresenta mecanismos de defesa e práticas de mitigação e, por fim, discute tendências arquiteturais voltadas a sistemas de larga escala, ambientes de integração contínua e uso de inteligência artificial para detectar falhas.

2. Revisão Bibliográfica

2.1. Fundamentos da Arquitetura Web

A Arquitetura Orientada a Serviços (SOA) é um modelo de organização de sistemas distribuídos baseado na interação entre três entidades: o cliente, que requisita um serviço, o provedor, que implementa e disponibiliza o serviço, e o registro de serviços, que atua como diretório para descoberta e localização dos serviços disponíveis [Mello et al. 2006]. A comunicação entre essas entidades segue o ciclo de (*publish-find-bind*) que consiste em publicar, descobrir e vincular, no qual o provedor publica a descrição do serviço no registro, o cliente consulta esse registro para localizar o serviço desejado e então estabelece uma conexão direta com o provedor. Para viabilizar esse modelo, três padrões são centrais: o SOAP (*Simple Object Access Protocol*), que define o formato das mensagens XML trocadas entre serviços; o WSDL (*Web Services Description Language*), que fornece uma descrição padronizada da interface do serviço; e o UDDI (*Universal Description, Discovery and Integration*), que implementa o diretório de registro e descoberta [Mello et al. 2006].

Com a evolução das aplicações web, essa arquitetura deu lugar a abordagens mais granulares, como os microsserviços, que fragmentam funcionalidades em serviços independentes comunicando-se por protocolos mais leves, como o REST [Pahl and Jamshidi 2016]. Essa transição arquitetural, discutida na Seção 4.3, trouxe novos desafios de segurança.

2.2. Conceitos de Segurança

A segurança da informação em aplicações web se fundamenta em cinco requisitos: confidencialidade, que assegura que os dados sejam acessíveis apenas por partes autorizadas; integridade, que garante que as informações não sejam alteradas indevidamente durante o armazenamento ou transmissão; disponibilidade, que exige que os serviços permaneçam acessíveis mesmo sob condições adversas; autenticidade, que permite verificar a identidade dos usuários e serviços envolvidos; e não-repúdio, que impede que uma entidade negue a autoria de uma ação realizada [Mello et al. 2006]. Na prática, esses requisitos se traduzem em mecanismos como criptografia de dados em trânsito e em repouso, controles

de autenticação e autorização, e estratégias de proteção contra negação de serviço. A maioria das aplicações web que processam entradas de usuários, trafegam dados sensíveis o que torna esses sistemas vulneráveis a ataques [Yadav et al. 2018]. Para organizar e classificar essas vulnerabilidades, a comunidade de segurança desenvolveu taxonomias de referência, o OWASP *Top 10* categoriza as dez classes de vulnerabilidades mais críticas em aplicações web [None 2008], e também existem classificações sistemáticas voltadas ao monitoramento de exploração de falhas [Shahriar and Zulkernine 2011]. A Seção 4.1 apresenta essa taxonomia em detalhe.

2.3. Vulnerabilidades e Mecanismos de Defesa

As vulnerabilidades em aplicações web podem ser agrupadas em categorias conforme o vetor de ataque explorado. As falhas de injeção, por exemplo, ocorrem quando dados fornecidos pelo usuário são interpretados como comandos ou consultas pelo sistema, permitindo a execução de operações não autorizadas. Na injeção de SQL (*SQLi*) o atacante manipula consultas ao banco de dados [Halfond et al. 2006].

Já no *Cross-Site Scripting* (XSS), os *scripts* maliciosos são inseridos em páginas web e executados no navegador de outros usuários [Gupta and Gupta 2015]. Outra categoria envolve falhas de controle de acesso, nas quais a aplicação não restringe adequadamente as ações ou recursos disponíveis a cada usuário, e ataques como o *Cross-Site Request Forgery* (CSRF) permitem a exploração de sessões autenticadas para executar requisições indevidas. Problemas de configuração incorreta e vulnerabilidades em componentes de terceiros também representam vetores frequentes [Alwan and Younis 2021].

Para mitigar essas ameaças, as estratégias de defesa atuam em diferentes camadas. No nível preventivo, técnicas como validação e sanitização de entradas, parametrização de consultas e uso de *tokens* anti-CSRF buscam eliminar as vulnerabilidades na origem [Shar and Tan 2016]. No nível de análise, ferramentas de teste de segurança, como SAST (*Static Application Security Testing*), DAST (*Dynamic Application Security Testing*) e IAST (*Interactive Application Security Testing*) permitem identificar falhas em diferentes estágios do ciclo de desenvolvimento. No nível de execução, soluções como WAF (*Web Application Firewall*) e RASP (*Runtime Application Self-Protection*) monitoram e protegem a aplicação em tempo real contra tentativas de exploração. A Seção 4.2 aprofunda a análise dessas ferramentas e técnicas.

3. Metodologia

REVISAR TODO MUNDO JUNTO

O levantamento bibliográfico foi realizado via *Google Scholar* com base na bibliografia sugerida, priorizando artigos de periódicos e anais de conferências da área. Ferramentas de IA Generativa **citar ferramentas usadas** foram empregadas como suporte na síntese inicial de conceitos e estruturação da visão geral do tema, para auxiliar a redação.

A redação foi dividida entre os autores, que trabalharam individualmente no desenvolvimento dos tópicos e posteriormente discutiram o texto final, sugerindo melhorias.

Toda a fundamentação técnica e o desenvolvimento do projeto foram validados e conduzidos pelos autores, garantindo o rigor científico e a integridade dos dados.

4. Discussão

Entre os principais conceitos utilizados para estruturar as vulnerabilidades e métodos de mitigação estão a taxonomia de vulnerabilidades, os mecanismos de segurança e as arquiteturas de segurança. Esses três elementos possuem papéis complementares e inter-relacionados no processo de identificação, prevenção e mitigação de vulnerabilidades em sistemas computacionais.

4.1. Taxonomia de Vulnerabilidades

A taxonomia de vulnerabilidades permite organizar falhas de segurança em categorias estruturadas, facilitando tanto a análise quanto a mitigação desses problemas. Nesse contexto, diferentes *frameworks* e organizações especializadas têm desenvolvido modelos de classificação amplamente utilizados pela comunidade de segurança da informação. Entre eles, destaca-se o trabalho da *OWASP Foundation*, que mantém uma das referências mais utilizadas para identificação e priorização de vulnerabilidades em aplicações web.

Os mecanismos de segurança, por sua vez, correspondem às técnicas, ferramentas e controles utilizados para prevenir, detectar ou mitigar vulnerabilidades. Esses mecanismos incluem práticas como validação de entrada de dados, autenticação multifator, criptografia, controle de acesso, filtragem de entrada e monitoramento de *logs*.

Já as arquiteturas de segurança referem-se à forma como os mecanismos de segurança são organizados e integrados dentro da estrutura de um sistema ou aplicação. A arquitetura define como diferentes componentes de segurança interagem entre si, estabelecendo camadas de proteção que contribuem para a defesa do sistema como um todo. Modelos arquiteturais como **defense in depth** e arquiteturas baseadas em microsserviços com isolamento de componentes são exemplos de estratégias utilizadas para fortalecer a segurança de aplicações modernas.

4.1.1. Framework OWASP Top 10: evolução 2017 → 2021

Ao longo desse período tem sido trocado o ranking das vulnerabilidades web, algumas tem ganhado mais espaço com ataques bem-sucedidos, como é o caso da Quebra de controle de Acesso que está em primeiro lugar, mostrando que há uma necessidade de controles mais rigorosos de autorização e não depende apenas da correção de falhas de implementação, mas sim adoção de princípios de segurança ao longo de todo o ciclo de vida do desenvolvimento.

As falhas criptográficas subiram para segunda posição, uma vulnerabilidade que muitas vezes leva à exposição de dados confidenciais ou sistemas comprometidos. A injeção que em 2017 estava em primeiro lugar, perdeu espaço e, em 2021 foi rebaixada para a terceira posição com uma taxa de incidência máxima de 19%, uma taxa de incidência média de 3,37%. Houve a criação de novos grupos, como Design Inseguro, que está em quarto lugar destacando problemas estruturais de arquitetura e modelagem de segurança desde as fases iniciais do desenvolvimento, como por exemplo, confiar apenas em verificações no *frontend*, ao invés de validar no servidor [None 2008].

4.1.2. Vulnerabilidades de Injeção (*SQLi, Command, LDAP*)

As vulnerabilidades de injeção constituem uma das classes mais conhecidas e exploradas em aplicações web. Elas ocorrem quando o usuário (cliente) fornece dados que são interpretados pelo servidor como parte de comando ou consultas executadas por sistemas subjacentes, como banco de dados ou sistemas operacionais. Para mitigar esse tipo de ataque deve-se usar uma validação rigorosa das entradas do usuário, uso de bibliotecas seguras de acesso (por exemplo, *Object Relational Mapping ORM*), princípio do menor privilégio e monitoramento contínuo de *logs* de acesso e execução.

Começando pelo *SQL injection (SQLi)* é uma vulnerabilidade que o usuário utiliza comandos SQL, (por exemplo, `SELECT * FROM <tabela> WHERE <coluna>=' ' OR 1=1`) para acessar, modificar ou até excluir dados sensíveis do banco de dados daquela aplicação web. O comando `1=1` é uma tautologia, ou seja, a condição sempre será verdadeira, ignorando a condição original na cláusula *WHERE* e retornando todas as linhas da tabela [Appelt 2014].

Outro tipo de ataque de injeção importante também é o *command injection*, uma vulnerabilidade em que a aplicação permite que dados fornecidos por um usuário sejam incorporados diretamente em comandos executados pelo sistema operacional. Isso pode levar a acesso ou modificação de arquivos sensíveis, escalada de privilégios, instalação de *malwares* ou *backdoors* e até o comprometimento completo do servidor.

A vulnerabilidade de *LDAP injection* ocorre quando entradas fornecidas por usuários são inseridas diretamente em consultas *Lightweight Directory Access Protocol* (LDAP) sem validação ou sanitização adequada. Dessa forma, um atacante pode manipular a estrutura de consulta enviada ao diretório, alterando sua lógica e obtendo acesso a informações não autorizadas.

4.1.3. *Cross-Site Scripting (XSS): Reflected, Stored, DOM-based*

O XSS é uma vulnerabilidade de segurança que permite a um atacante inserir *scripts* maliciosos em páginas web visualizadas por outros usuários. É comum que dados fornecidos por usuários sejam exibidos posteriormente em páginas *HTML*. Quando esses dados não passam por validação, *scripts* inseridos por atacantes podem ser interpretados pelo navegador como código legítimo [Sarmah et al. 2018].

Reflected XSS ocorre quando o código malicioso enviado pelo atacante é imediatamente refletido na resposta da aplicação web. Normalmente, esse ataque ocorre em parâmetros de requisições HTTP, como campos de busca ou parâmetros de URL.

Stored XSS ocorre quando o código malicioso é armazenado permanentemente no servidor, como em comentários de blogs, fóruns, perfis de usuário e sistemas de mensagens. Quando outros usuários forem acessar o conteúdo armazenado, o *script* é executado automaticamente em seus navegadores. Ele é potencialmente mais perigoso porque não depende de interação com o atacante, podendo afetar múltiplos usuários.

DOM-based XSS ocorre quando a vulnerabilidade está presente no código *JavaScript* executado no lado do cliente. Nesse caso, a manipulação insegura do *Document Ob-*

ject Model (DOM) permite que dados provenientes da URL ou de outras fontes sejam interpretados como código executável. Diferente dos outros tipos, nesse caso a modificação ocorre inteiramente no navegador da vítima, sem que o servidor necessariamente processe o conteúdo malicioso.

Os impactos podem ser diversos, como roubo de *cookies* de sessão, sequestro de sessões autenticadas, redirecionamento para páginas maliciosas, execução de ações em nome do usuário, distribuição de *malware*.

4.1.4. Cross-Site Request Forgery (CSRF)

O *Cross-Site Request Forgery (CSRF)* é um ataque que explora a confiança que uma aplicação web deposita no navegador do usuário (cliente) autenticado. Nesse tipo de ataque, o invasor induz a vítima a executar uma requisição maliciosa em um site no qual ela já está autenticada, fazendo com que o servidor interprete essa ação como legítima.

Funcionando da seguinte maneira, quando um usuário realiza login em um site, o servidor geralmente mantém sua sessão ativa por meio de *cookies* de autenticação (usando um *token JWT*). Se o usuário visitar um site malicioso enquanto está autenticado, esse site pode gerar uma requisição automática para o site legítimo. Como o navegador envia automaticamente os *cookies* associados ao domínio, o servidor interpreta a requisição como se tivesse sido realizada pelo próprio usuário [Lin et al. 2009].

4.1.5. Broken Access Control e IDOR

O *Broken Access Control* é uma vulnerabilidade que ocorre quando uma aplicação falha em aplicar corretamente as regras de autorização, permitindo que usuários executem ações ou acessem recursos que deveriam estar restritos. Esse tipo de falha está associado a sistemas em que os mecanismos de verificação de permissões são implementados de forma inadequada ou incompleta [Kusnana et al. 2025].

Considerando um *endpoint* administrativo como por exemplo `https://site.com/admin/financas`, se o servidor não validar corretamente as permissões associadas à sessão do usuário, ele poderá acessar as finanças mesmo sem possuir privilégios adequados.

O *Insecure Direct Object Reference (IDOR)* é uma vulnerabilidade que ocorre quando uma aplicação permite que usuários acessem diretamente objetos ou recursos internos por meio de identificadores únicos fornecidos na requisição, sem verificar adequadamente se o usuário possui autorização para acessar aquele recurso específico.

Na grande parte dos sistemas web, recursos como perfis de usuário, pedidos, documentos, entre outros, são identificados por *IDs*, números ou strings únicas. Quando a aplicação utiliza esses identificadores diretamente na URL ou em parâmetros de requisição sem realizar verificações de autorização, um atacante pode manipulá-los para acessar informações de outros usuários.

Exemplificando, o site `https://site.com/pedido?id=1025`, o parâmetro está recebendo um ID de pedido, se o servidor não fizer a verificação de autenticação, um atacante

pode alterar o ID do pedido e ter acesso aos pedidos pertencente a outros usuários.

4.1.6. Falhas Criptográficas

As falhas criptográficas referem-se ao uso incorreto, insuficiente ou inadequado de mecanismos criptográficos para proteger os dados sensíveis em sistemas computacionais. Uma das causas mais comuns de falhas criptográficas é o uso de algoritmos obsoletos ou considerados inseguros. Algoritmos como SHA-1(para hashing), DES ou 3DES (para cifragem), entre outros, já não oferecem garantias adequadas contra ataques modernos, devido à possibilidade de colisões e o avanço de processamento de hardwares modernos. Além disso, a utilização incorreta de algoritmos considerados seguros como o emprego de criptografia simétrica sem gerenciamento adequado de chaves, geração previsível de números aleatórios ou reutilização de vetores de inicialização (IV), podem comprometer completamente a segurança dos sistemas [Blessing et al. 2021].

Outro problema recorrente é o armazenamento de senhas em texto puro (plain text) ou com técnicas inadequadas de hash, sem o uso de sal ou funções específicas para armazenamento seguro de credenciais.

Falhas também podem ocorrer durante a transmissão de dados, especialmente quando protocolos seguros não são utilizados ou são configurados incorretamente. A ausência de HTTPS ou o uso inadequado de certificados digitais pode expor informações sensíveis a ataque de interceptação conhecido como main-in-the-middle (MitM).

4.1.7. *Misconfiguration e Supply Chain*

As más configurações (*misconfiguration*) ocorrem quando sistemas são implantados com definições inseguras, seja por falta de conhecimento técnico, ausência de políticas padronizadas ou por erro humano. Exemplos clássicos incluem *buckets* de armazenamento em nuvem com permissões de leitura pública, portas de administração expostas à internet, banco de dados sem autenticação adequada, senhas com valores padrões e etc. De acordo com relatórios da *Cloud Security Alliance*, as más configurações representam uma das principais causas de violação de dados em ambiente *cloud* [Cloud Security Alliance 2024].

Os impactos são diversos como acesso não autorizado aos sistemas, vazamento de informações sensíveis, execução de ataques automatizados e até o comprometimento completo da aplicação ou servidor.

Já o ataque *Supply chain* tem se tornado cada vez mais sofisticado e frequente. Estes ataques exploram as relações de confiança estabelecidas entre organizações e seus fornecedores, bibliotecas de código aberto, *frameworks* e pipelines de integração contínua. Um invasor pode comprometer um componente amplamente utilizado e, através dele, infectar outras aplicações que dependem daquele componente. Como exemplo notório o incidente em 2018 no qual foi inserido um código malicioso na biblioteca *event-stream* no NPM (gerenciador de pacotes Node), o código inserido teve como objetivo coletar a chave privada da carteira de criptomoedas, criptografar com uma chave pública embutida e enviar via HTTP para servidores remotos.

Esse exemplo ilustra os danos gerados, como execução de código malicioso dentro de aplicações legítimas, roubo de dados sensíveis, comprometimento em larga escala de múltiplos sistemas e distribuição de *malware* para usuários finais.

4.2. Mecanismos de Defesa

Os mecanismos de defesa complementam a taxonomia de vulnerabilidades ao fornecer técnicas, ferramentas e controles concretos para prevenir, detectar ou mitigar ataques. Esta seção organiza as defesas por categoria de ataque, apresenta as ferramentas de análise estática e dinâmica, e discute as soluções de proteção em tempo de execução (*runtime*).

4.2.1. Defesas por categoria

Cada categoria de vulnerabilidade descrita nas seções anteriores possui mecanismos de defesa específicos, com diferentes níveis de efetividade e limitações. A Tabela 1 consolida a relação entre técnicas de defesa, ataques mitigados e suas restrições, com base em trabalhos de classificação como os de [Halfond et al. 2006], [Gupta and Gupta 2015], [Barth et al. 2022] e [None 2008].

4.2.2. Ferramentas de análise: SAST, DAST, IAST

As ferramentas de análise de segurança identificam vulnerabilidades ao longo do ciclo de vida da aplicação. O SAST (*Static Application Security Testing*) analisa código-fonte ou *bytecode* sem execução, sendo adequado para integração em pipelines CI/CD [Alazmi and De Leon 2022]. O DAST (*Dynamic Application Security Testing*) realiza testes em tempo de execução, simulando ataques à aplicação, com avanços como a automação de autenticação via OWASP ZAP (Sacramento et al., 2024). Já o IAST (*Interactive Application Security Testing*) combina abordagens estática e dinâmica por meio de instrumentação durante testes funcionais, aumentando a precisão e reduzindo falsos positivos [Cotroneo 2025]. Evidências recentes reforçam a importância da combinação dessas técnicas para uma cobertura mais eficaz [De Jesús Domínguez-García et al. 2023].

4.2.3. Defesas em *runtime*: WAF, WAAP, RASP

Defesas em *runtime* atuam em produção para proteger a aplicação em execução. O WAF (*Web Application Firewall*) inspeciona o tráfego HTTP e bloqueia requisições maliciosas com base em regras e assinaturas, embora enfrente limitações semelhantes às de ferramentas *black-box* [Doupé et al. 2010]. O WAAP (*Web Application and API Protection*) amplia essa abordagem ao incluir proteção de APIs e mitigação de ataques modernos. Já o RASP (*Runtime Application Self-Protection*) atua dentro da aplicação, utilizando o contexto de execução para aumentar a precisão, reduzir falsos positivos e mitigar ataques *zero-day* [Sureda Riera et al. 2025].

Tabela 1. Técnicas de defesa por categoria de ataque e limitações

Técnica	Ataque	Limitações
Prepared Statements/ ORM	Injeção (SQLi; LDAP)	Requer refatoração; não cobre lógica de negócio vulnerável
Validação positiva de entrada	Injeção (SQLi; LDAP; Command)	Listas de permissão podem ficar desatualizadas; bypass por encoding
Content Security Policy (CSP)	XSS (Reflected; Stored; DOM)	Configuração complexa; incompatibilidade com bibliotecas legadas
Encoding contextual/ Sanitização	XSS	Dependente do contexto (HTML; JS; URL); mXSS em innerHTML [Heiderich et al. 2013]
HTTPOOnly/ Secure cookies	Roubo de sessão via XSS	Não impede CSRF; XSS ainda pode executar ações em nome do usuário
CSRF Token/ Same-Site/ Double Submit	CSRF	Token em APIs stateless; SameSite=Lax não cobre GET maliciosos
RBAC/ ABAC/ Validação backend	Broken Access Control; IDOR	Modelagem de permissões complexa; validação deve ser consistente
TLS 1.3/ bcrypt-Argon2/ Criptografia em repouso	Falhas criptográficas	Configuração incorreta; algoritmos fracos em sistemas legados
IaC scanning/ SBOM/ Lockfiles	Misconfiguration; Supply Chain	Cobertura parcial; dependências transitivas

Fonte: [Halfond et al. 2006]; [Heiderich et al. 2013]; [Gupta and Gupta 2015]; [Barth et al. 2022]; [None 2008].

Tabela 2. Comparativo SAST, DAST, IAST

Critério	SAST	DAST	IAST
Momento da análise	Pré-execução (código)	Runtime (aplicação em execução)	Durante testes funcionais
Cobertura de fluxo	Parcial (sem contexto runtime)	Completa (tráfego real)	Alta (instrumentação)
Falsos positivos	Maior	Intermediário	Menor [Cotroneo 2025]
Integração CI/CD	Nativa	Requer ambiente implantado (deployed)	Requer agente no processo

Fonte: [Alazmi and De Leon 2022]; [Sacramento et al. 2024]; [De Jesús Domínguez-García et al. 2023]; [Cotroneo 2025].

4.2.4. Comparativo WAF x RASP

[Sureda Riera et al. 2025] realizaram uma análise sistemática da efetividade de ferramentas de proteção em *runtime*, comparando WAF e RASP em diferentes categorias de ataque. Os resultados indicam que o RASP supera o WAF em precisão por categoria de ataque, em parte devido ao conhecimento contextual da aplicação. A Tabela 3 sintetiza o comparativo empírico com base nesse estudo.

Tabela 3. Comparativo empírico WAF x RASP

Categoria de ataque	WAF (efetividade)	RASP (efetividade)	Observação
SQL Injection	Média (assinaturas)	Alta (contexto de query)	RASP identifica ponto de injeção
XSS	Média (padrões HTML/JS)	Alta (contexto de saída)	RASP conhece fluxo de dados
Command Injection	Variável	Alta	RASP monitora chamadas de sistema
Path Traversal	Média	Alta	RASP intercepta acesso a arquivos
Zero-day/variações	Baixa	Maior resiliência	RASP baseado em comportamento

Fonte: [Alazmi and De Leon 2022]; [Sacramento et al. 2024]; [De Jesús Domínguez-García et al. 2023]; [Cotroneo 2025].

Os autores recomendam o uso complementar de WAF e RASP: o WAF na borda para filtrar tráfego conhecido e mitigar DDoS, e o RASP para proteção contextual contra explorações que ultrapassem a camada perimetral.

4.3. Segurança de Arquiteturas Modernas

A evolução dos sistemas web para arquiteturas modernas, como microsserviços e aplicações baseadas em nuvem, transformou significativamente a superfície de ataque. Com a fragmentação de aplicações monolíticas em Serviços Web distribuídos, a comunicação inter-processos e a exposição de APIs tornaram-se o núcleo do funcionamento e, conseqüentemente, da segurança destas aplicações.

4.3.1. APIs REST — OWASP API Top 10, BOLA, *rate limiting*

APIs REST são extremamente comuns hoje em dia, fazendo parte de muitas aplicações modernas. Com base no OWASAP API Top 10 de 2023, que seria a lista mais atualizada das vulnerabilidades mais críticas de APIs, a vulnerabilidade mais comum para essas APIs é *Broken Object Level Authorization* (BOLA), um tipo de vulnerabilidade muito difundida e de fácil detecção. Esse problema ocorre pela falha na validação de permissões no nível de objeto, em que APIs expõem um identificador direto e não verifica no backend se o usuário logado tem autorização para acessar aquele registro, podendo deixar um usuário não autorizado ler, alterar ou deletar dados de outra pessoa.

Uma técnica de defesa como *rate limiting*, quando em falta pode ocasionar em vulnerabilidades também, como o Consumo Irrestrito de Recursos (*Unrestricted Resource Consumption*), a vulnerabilidade top 4 na lista de 2023. O processamento de requisições consome recursos do servidor, ou até recursos terceiros, que se não limitados por quantidade ou frequência, pode fazer a API vulnerável.

4.3.2. APIs GraphQL — introspecção, *query depth*, DoS

O modelo GraphQL é uma alternativa a APIs RESTful, e com uma ampla adoção, também tem desafios de segurança exclusivos. O GraphQL entrega ao cliente a capacidade de formular a complexidade da requisição. Nesse modelo, a comunicação flui por um único endpoint (geralmente `/graphql`), e com isso, o problema é que uma requisição sintaticamente válida pode ser projetada para exigir um esforço massivo do servidor, esse seria o DoS (Denial of Service) em GraphQL. Se o servidor não estiver otimizado, pode fazer com que uma única chamada frontend realize centenas de consultas ao banco de dados. A *query depth* é o principal vetor utilizado para causar o DoS. Ela se refere ao nível de aninhamento dos campos da estrutura da requisição. A solução é validar a estrutura da requisição. Uma prática padrão é utilizar analisadores que verificam a *Abstract Syntax Tree* (AST) da query assumindo que ela chega.

A introspecção é uma funcionalidade nativa e muito poderosa do GraphQL que permite ao cliente consultar o próprio servidor para descobrir o que ele é capaz de fazer. O perigo disso é deixar a introspecção ativada em produção, que equivaleria a entregar ao atacante a documentação completa da sua API. Isso revela a estrutura exata dos relacionamentos, deixando possível ataques precisos de BOLA (Broken Object Level Authorization) ou elaborar queries profundas para derrubar o servidor. A recomendação unânime de segurança é desativar as consultas de introspecção no ambiente de produção.

4.3.3. Microserviços — mTLS, API Gateway, gestão de segredos

O artigo "Zero Trust Security Architecture in Microservices via mTLS" [IJARCSE, 2025], propõe um modelo de segurança focado em eliminar a confiança implícita em redes distribuídas.

O mTLS (Mutual Transport Layer Security) atua como pilar fundamental da arquitetura Zero Trust (ZTSA). Diferente do TLS comum, ele garante que as comunicações entre os microserviços sejam não apenas criptografadas, mas autenticadas de forma mútua. proxies sidecar (Envoy) são acoplados a cada microserviço e encarregados de validar os certificados dos pares a cada nova conexão estabelecida. O API Gateway funciona como o principal ponto de controle de acesso para o tráfego externo que entra na aplicação. O API Gateway se comunica diretamente com o Provedor de Identidade (como o Keycloak) para validar os tokens (JWTs) recebidos dos usuários. Somente após a validação bem-sucedida, o API Gateway repassa o tráfego para os microserviços internos através de conexões protegidas por mTLS. A arquitetura aborda a gestão de segredos por meio de **Rotação Agressiva de Certificados e Tokens de Curta Duração**.

A Rotação Agressiva de Certificados em vez de usar certificados estáticos, o com-

ponente Citadel do Istio emite automaticamente certificados X.509 com um período de validade muito curto, de apenas 24 horas. Para estreitar ainda mais a janela de oportunidade para atacantes, o sistema força a rotação desses certificados a cada 12 horas. Já os Tokens de Curta Duração utiliza JSON Web Tokens (JWTs) emitidos de forma centralizada para representar identidades. O documento alerta que o uso de tokens de curta duração minimiza os riscos de segurança, mas exige mecanismos muito eficientes para atualização.

4.3.4. Cloud-native — IaC scanning, Kubernetes, S3

Aplicações nativas da nuvem dependem ativamente de infraestruturas altamente dinâmicas e configuráveis (orquestradores de contêineres e armazenamento em provedores). O uso excessivo de recursos sem as restrições adequadas frequentemente leva a vulnerabilidades de Má Configuração de Segurança. Um cenário de ataque comum envolve permissões padrão em provedores de nuvem (CSP) deixadas abertas para a Internet (por exemplo, permissões de armazenamento do Amazon S3). Isso permite que dados confidenciais sejam facilmente acessados por qualquer usuário da rede. A prevenção exige processos de *hardening* consistentes e a revisão constante das permissões de armazenamento em nuvem (ACLs e *buckets*).

Por fim, o ecossistema *cloud-native* faz forte uso de Infraestrutura como Código (IaC) e *pipelines* de CI/CD (Integração e Entrega Contínuas). O OWASP introduziu a categoria "Falhas na Integridade de Dados e Software" (A08:2021), focando em atualizações e *pipelines* não verificados. Sem o devido controle, *pipelines* de CI/CD podem introduzir acessos não autorizados ou implantar códigos maliciosos na infraestrutura. A defesa engloba a configuração e o controle de acesso restrito aos *pipelines*, verificação de dependências com ferramentas de *Software Composition Analysis* e a implementação de verificações focadas em Infraestrutura como Código (IaC) para prevenir vulnerabilidades de provisionamento.

5. Tendências e Pesquisas em Andamento ANA TEM QUE REVISAR

As tecnologias de desenvolvimento evoluem na mesma medida que evoluem as ameaças a aplicações web, o que impulsiona as pesquisas voltadas a novas abordagens para a proteção de aplicações web. A mitigação de ataques complexos hoje exige que a segurança seja incorporada de forma pervasiva, desde o design até a execução. A seguir, mencionamos algumas das principais tendências atuais.

5.1. Inteligência Artificial para Detecção de Ameaças

A aplicação de técnicas de inteligência artificial, em especial de aprendizado de máquina, tem revolucionado a detecção de vulnerabilidades e a classificação de anomalias maliciosas.

Abordagens modernas utilizam arquiteturas avançadas de redes neurais, como redes neurais convolucionais (CNNs), redes neurais recorrentes (LSTM) e redes neurais em grafos (GNN), para analisar fluxos de dados complexos e identificar padrões sutis de injeção.

Pesquisas atuais utilizam técnicas como análise de N-Grams e *support vector machines* (SVM) para classificar tráfego e identificar vulnerabilidades (como injeções de XSS e SQL) com maior precisão.

Além disso, abordagens comportamentais baseadas em anomalias (inspiradas no sistema imunológico humano, por exemplo) têm sido implementadas no lado do servidor para mitigar ataques desconhecidos (*zero-day*), reduzindo a dependência exclusiva de assinaturas rígidas.

Paralelamente, pesquisas sobre modelos adversariais buscam fortalecer esses mesmos classificadores contra atacantes que tentam contornar os filtros de segurança através de pequenas mutações no *input* malicioso.

O uso de IA em sistemas de detecção de intrusão (IDS) levanta o problema da interpretabilidade, exigindo o avanço em modelos de inteligência artificial explicáveis para que profissionais consigam auditar por que uma requisição foi bloqueada.

5.2. Arquiteturas de Confiança Zero e Gestão Avançada de Identidades

À medida que arquiteturas baseadas em microsserviços e APIs distribuídas se tornam a norma, a adoção de Arquiteturas de Confiança Zero (*Zero Trust*) é fundamental. Sob essa premissa, nenhuma requisição ou transação é confiável por padrão, mesmo que originada dentro da rede corporativa, exigindo verificação e autorização contínuas das identidades (federadas ou não) e contexto dos dispositivos.

Aplicações, mesmo que internas, frequentemente cruzam múltiplos domínios de segurança, exigindo gestão de identidades federadas e o estabelecimento dinâmico de confiança. Tecnologias de padronização, como o SAML 2.0 e XACML, têm evoluído para suportar autenticação única (SSO) com o uso de pseudônimos e identificadores opacos.

Essa abordagem não apenas garante autorização baseada em atributos de forma interoperável, mas também protege a privacidade do usuário contra o rastreamento indevido.

5.3. DevSecOps - Integração de segurança no CI/CD

Para mitigar falhas ainda em fase de código, a indústria tem adotado a esteira de *DevSecOps*, onde integra-se a automação rigorosa de verificações de integridade nativamente no pipeline de integração e entrega contínuas (CI/CD). Isso assegura que softwares atualizados ou componentes de bibliotecas de terceiros sejam validados contra vulnerabilidades antes de alcançarem o ambiente de produção.

Esse processo de integração contínua da segurança no ciclo de vida de desenvolvimento é uma resposta à demanda por entregas ágeis.

A incorporação de ferramentas de testes estáticos, dinâmicos e interativos, (SAST, DAST e IAST) nos processos de CI/CD permite descobrir vulnerabilidades de injeção e falhas lógicas antes que o software chegue à produção.

Adicionalmente, com a introdução da categoria de "design inseguro" na OWASP Top 10 de 2021, o mercado reforça a necessidade de preocupar-se com segurança desde o início (o que é chamado de *shift-left*) usando de padrões de design seguros e arquiteturas de referência desde as fases pré-código.

5.4. Segurança em Cadeias de Suprimento e Componentes de Terceiros

O uso massivo de bibliotecas, repositórios de código aberto e redes de distribuição de conteúdo (CDNs) levanta preocupações severas de integridade. As pesquisas em andamento focam no monitoramento de dependências e verificação de integridade de software por meio de ferramentas como OWASP *dependency check* ou assinaturas digitais, garantindo que o pipeline de CI/CD e as atualizações automáticas não introduzam vulnerabilidades legadas ou código malicioso de fornecedores comprometido.

5.5. Desafios Abertos

Apesar da evolução, diversos desafios técnicos permanecem abertos para a comunidade de segurança.

Os modelos de comunicação modernos, orientados fortemente a APIs (REST, GraphQL) e processamento no lado do cliente (DOM-based), mudam a superfície de ataque clássica. A proliferação das *single-page applications* (SPAs) introduz complexidades significativas no lado do cliente, dificultando o rastreamento e mitigação de vulnerabilidades puramente baseadas no manipulador de DOM (como o DOM-based XSS). O controle do DOM e a sanitização contextual no navegador são hoje desafios cruciais para conter o XSS baseado em DOM.

Ao mesmo tempo, a descoberta automatizada e o monitoramento de APIs ocultas (*shadow APIs*), que resultam em pontos não mapeados de possível vulnerabilidade de um sistema, exigem controles refinados de permissões e de *rate limit*, já que os mecanismos de segurança baseados no cliente podem ser facilmente contornados por atacantes diretos na API.

6. Conclusão

7. Conclusão

REVISAR TODO MUNDO JUNTO

Este trabalho demonstrou que a segurança em aplicações web consolidou-se como um pilar estratégico, transcendendo a camada perimetral de rede [Cloud Security Alliance 2024, Alwan and Younis 2021]. A defesa em profundidade surge como o princípio unificador fundamental, integrando a análise de código (SAST/IAST), testes dinâmicos (DAST) e proteção em *runtime* (RASP) para mitigar falhas desde o *design* até a execução [De Jesús Domínguez-García et al. 2023, Alsaedi 2023, Barth et al. 2022, Pahl and Jamshidi 2016].

Apesar dos avanços, identificou-se uma lacuna na literatura e na prática atual quanto à segurança de arquiteturas descentralizadas, como *shadow APIs* e o processamento complexo no lado do cliente (*DOM-based*), que dificultam o rastreamento automatizado de vulnerabilidades [Escape 2023, Tanveer 2025, Sureda Riera et al. 2025]. Além disso, embora a Inteligência Artificial (IA) seja promissora na detecção de anomalias, sua falta de interpretabilidade ainda limita a auditoria crítica em sistemas de larga escala [Yadav 2025a, Alsaedi 2023].

Como direções futuras, aponta-se a necessidade de amadurecer modelos de IA explicável aplicados à cibersegurança e a consolidação de arquiteturas *Zero Trust* baseadas

em identidades dinâmicas [Yadav 2025a, of Standards and (NIST) 2020, Yadav 2025b]. A evolução para uma cultura *DevSecOps* plenamente automatizada é essencial para garantir que a agilidade no desenvolvimento não comprometa a integridade dos dados e a resiliência das infraestruturas modernas [Edmundson and Hartman 2022, De Jesús Domínguez-García et al. 2023].

8. Referencias TODO MUNDO TEM QUE REVISAR

Referências

- Alazmi, S. and De Leon, D. C. (2022). A systematic literature review on the characteristics and effectiveness of web application vulnerability scanners. *IEEE Access*, 10:33200–33219.
- Alsaedi, M. (2023). Deep learning technique-enabled web application firewall for the detection of web attacks. *Sensors*, 23(4):2073.
- Alwan, Z. S. and Younis, M. F. (2021). *An OWASP top ten driven survey on web application protection methods*, page 203–222. Springer.
- Appelt, D. (2014). Automated testing for sql injection vulnerabilities: an input mutation approach. In *International Symposium on Software Testing and Analysis (ISSTA)*, page 259–269, San Jose. ACM.
- Barth, A., Jackson, C., and Mitchell, J. C. (2022). Systematic approach for web protection runtime tools’ effectiveness analysis. In *ACM Conference on Computer and Communications Security (CCS)*, page 75–88, Alexandria. ACM.
- Blessing, J., Specter, M. A., and Weitzner, D. J. (2021). You really shouldn’t roll your own crypto: An empirical study of vulnerabilities in cryptographic libraries.
- Cloud Security Alliance (2024). Top threats to cloud computing. Technical report, Cloud Security Alliance (CSA). Acessado em: 16 mar. 2026.
- Cotroneo, D. (2025). Comparing effectiveness and efficiency of interactive application security testing (iast) and runtime application self-protection (rasp) tools in a large java-based system. *Empirical Software Engineering*.
- De Jesús Domínguez-García, A., Limón, X., Ocharán-Hernández, J. O., and Pérez-Arriaga, J. C. (2023). Security testing for web applications: A systematic literature review. In *2023 11th International Conference in Software Engineering Research and Innovation (CONISOFT)*, page 82–91.
- Doupé, A., Cova, M., and Vigna, G. (2010). *Detection of Intrusions and Malware, and Vulnerability Assessment*, volume 6201, page 111–131. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Edmundson, C. and Hartman, K. G. (2022). *SANS 2022 DevSecOps survey: creating a culture to significantly improve your organization’s security posture*.
- Escape (2023). The state of graphql security 2023.
- Gupta, S. and Gupta, B. B. (2015). Cross-site scripting (xss) attacks and defense mechanisms: classification and state-of-the-art. *International Journal of System Assurance Engineering and Management*, 8(S1):512–530.

- Halfond, W. G. J., Viegas, J., and Orso, A. (2006). A classification of sql-injection attacks and countermeasures. In *IEEE International Symposium on Secure Software Engineering (ISSSE)*. IEEE.
- Heiderich, M., Schwenk, J., Frosch, T., Magazinius, J., and Yang, E. Z. (2013). mxss attacks: Attacking well-secured web-applications by using innerhtml mutations. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security - CCS '13*, page 777–788, Berlin, Germany. ACM Press.
- Kusnana, K., Gymnastiar, P., and Riswanto, B. (2025). Analisis kerentanan insecure direct object reference (idor) dan broken access control pada aplikasi invoice berbasis web. *Digital Transformation Technology*, 5(2):106–114.
- Lin, X., Zavarisky, P., Ruhl, R., and Lindskog, D. (2009). Threat modeling for csrf attacks. In *2009 International Conference on Computational Science and Engineering*, volume 3, pages 486–491.
- Mello, E. R., Wangham, M. S., da Silva Fraga, J., and Camargo, E. (2006). *Segurança em Serviços Web*.
- None (2008). Robust defenses for cross-site request forgery.
- of Standards, N. I. and (NIST), T. (2020). *NIST special publication 800-207: zero trust architecture*. Gaithersburg.
- Pahl, C. and Jamshidi, P. (2016). Microservices: a systematic mapping study. In *International Conference on Cloud Computing and Services Science (CLOSER)*, page 137–146, Rome. SCITEPRESS.
- Sacramento, L., Ítalo Cunha, Cardoso, G., Souza, A., Franco, A., and Oliveira, L. (2024). Automação de autenticação para testes de segurança em aplicações web no zap. In *Anais do XXIV Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg 2024)*, page 760–766, Porto Alegre, RS, Brasil. Sociedade Brasileira de Computação - SBC.
- Sarmah, U., Bhattacharyya, D., and Kalita, J. (2018). A survey of detection methods for xss attacks. *Journal of Network and Computer Applications*, 118:113–143.
- Shahriar, H. and Zulkernine, M. (2011). Taxonomy and classification of automatic monitoring of program security vulnerability exploitations. *Journal of Systems and Software*, 84(2):379–393.
- Shar, L. K. and Tan, H. B. K. (2016). Securing web applications from injection and logic vulnerabilities: approaches and challenges. *Information and Software Technology*, 74:160–180.
- Sureda Riera, T., Ramón Bermejo Higuera, J., Bermejo Higuera, J., Antonio Sicilia Montalvo, J., and Martínez Herráiz, J. J. (2025). Twenty-two years since revealing cross-site scripting attacks: a systematic mapping and a comprehensive survey. *Computer Modeling in Engineering & Sciences*, 133(3):579–599.
- Tanveer, H. (2025). Towards secure apis: a survey on restful api vulnerability detection. *Computers, Materials & Continua*, 84(3).

- Yadav, D. (2025a). Cutting-edge approaches in intrusion detection systems: deep learning, reinforcement learning, and ensemble techniques. *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*.
- Yadav, D. (2025b). Zero trust security architecture in microservices via mTLS + OAuth 2.0. *International Journal of Advanced Research in Computer Science and Electronics Engineering*.
- Yadav, D., Gupta, D., Singh, D., Kumar, D., and Sharma, U. (2018). Vulnerabilities and security of web applications. In *2018 4th International Conference on Computing Communication and Automation (ICCCA)*, page 1–5.